

riko FAQ

Index

[What pipes are available](#) | [What file types are supported](#) | [What protocols are supported](#)

What pipes are available?

Overview

riko's available pipes are outlined below ¹:

Pipe name	Pipe type	Pipe sub-type	Pipe description
aggregate	operator	aggregator	performs an arbitrary (user-defined) function on a stream
count	operator	aggregator	counts the number of items in a feed
csv	processor	source	parses a csv file to yield items
currencyformat	processor	transformer	formats a number to a given currency string
dateformat	processor	transformer	formats a date
exchangerate	processor	transformer	retrieves the current exchange rate for a given currency pair
feedautodiscover	processor	source	fetches and parses the first feed found on a site
fetch	processor	source	fetches and parses a feed to return the entries
fetchdata	processor	source	fetches and parses an XML or JSON file to return the feed entries
fetchpage	processor	source	fetches the content of a given web site as a string
fetchsitefeed	processor	source	fetches and parses the first feed found on a site
fetchtext	processor	source	fetches and parses a text file
filter	operator	composer	extracts items matching the given rules
hash	processor	transformer	hashes the field of a feed item
input	processor	source	prompts for text and parses it into a variety of different types, e.g., int, bool, date, etc
itembuilder	processor	source	builds an item

join	operator	aggregator	perform a SQL like join on two feeds
regex	processor	transformer	replaces text in fields of a feed item using regexes
refind	processor	transformer	finds text located before, after, or between substrings using regular expressions
rename	processor	transformer	renames or copies fields in a feed item
reverse	operator	composer	reverses the order of source items in a feed
rssitembuilder	processor	source	builds an rss item
simplemath	processor	transformer	performs basic arithmetic, such as addition and subtraction
slugify	operator	transformer	slugifies text
sort	operator	composer	sorts a feed according to a specified key
split	operator	composer	splits a feed into identical copies
strconcat	processor	transformer	concatenates strings
strfind	processor	transformer	finds text located before, after, or between substrings
strreplace	processor	transformer	replaces the text of a field of a feed item
strtransform	processor	transformer	performs string transformations on the field of a feed item
subelement	processor	transformer	extracts sub-elements for the item of a feed
substr	processor	transformer	returns a substring of a field of a feed item
sum	operator	aggregator	sums a field of items in a feed
tail	operator	composer	truncates a feed to the last N items
timeout	operator	composer	returns items from a stream until a certain amount of time has passed
tokenizer	processor	transformer	splits a string by a delimiter
truncate	operator	composer	returns a specified number of items from a feed
udf	processor	transformer	performs an arbitrary (user-defined) function on an item
union	operator	composer	merges multiple feeds together

uniq	operator	composer	filters out non unique items according to a specified field
urlbuilder	processor	transformer	builds a url
urlparse	processor	transformer	parses a URL into its six components
xpathfetchpage	processor	source	fetches the content of a given website as DOM nodes or a string
yql	processor	source	fetches the result of a given YQL query

Args

riko pipes come in two flavors; operator and processor². operator`s operate on an entire ``stream at once. Example operator`s include ``pipecount, pipefilter, and pipereverse.

```
>>> from riko.modules.reverse import pipe
>>>
>>> stream = [{'title': 'riko pt. 1'}, {'title': 'riko pt. 2'}]
>>> next(pipe(stream))
{'title': 'riko pt. 2'}
```

processor`s process individual ``items. Example processor`s include ``pipefetchsitefeed, pipehash, pipeitembuilder, and piperegex.

```
>>> from riko.modules.hash import pipe
>>>
>>> item = {'title': 'riko pt. 1'}
>>> stream = pipe(item, field='title')
>>> next(stream)
{'title': 'riko pt. 1', 'hash': 2853617420L}
```

Kwargs

The following table outlines the available kwargs.

kwarg	type	description	default
conf	dict	The pipe configuration	varies
extract	str	The key with which to get a value from <i>conf</i>	None
listize	bool	Ensure that an <i>extract</i> value is list-like	False
pdictize	bool	Convert <i>conf</i> / <i>extract</i> to a DotDict instance	varies
objectify	bool	Convert <i>conf</i> to an Objectify instance	varies
ptype	str	Used to convert <i>conf</i> items to a specific type.	pass
dictize	bool	Convert the input <i>item</i> to a DotDict instance	True

field	str	The key with which to get a value from the input	None
ftype	str	Converts the input <i>item</i> to a specific type	pass
count	str	The output count	all
assign	str	Attribute used to assign output	varies
emit	bool	Return the output as is (don't assign)	varies
skip_if	func	Determines if processing should be skipped	None
inputs	dict	Values to be used in place of prompting the user	None

Notes

What file types are supported?

File types that riko supports are outlined below:

File type	Recognized extension(s)	Supported pipes
HTML	html	feedautodiscovery, fetchpage, fetchsitefeed
XML	xml	fetch, fetchdata
JSON	json	fetchdata
Comma separated file	csv, tsv	csv

What protocols are supported?

Protocols that riko supports are outlined below:

Protocol	example
http	http://google.com
https	https://github.com/reubano/feed
file	file:///Users/reubano/Downloads/feed.xml

-
- 1 See [Design Principles](#) for explanation on *pipe* types and sub-types
 - 2 See [Alternate workflow creation](#) for pipe composition examples